RELATÓRIO TÉCNICO DE GOVERNANÇA E ÉTICA DE IA

O Desafio Final: TaskAnalyzer — DevOps, Testes Automatizados, Containerização e Auditoria Ética (Fase 3)

Instituição: Centro Universitário de Brasília (CEUB) | **Disciplina:** Bootcamp III — Fase 3 (25,0 Pontos)
Aluno / Engenheiro Responsável: Victor Hugo Azevedo Alves Zainote (`@VictorZainote`)
Repositório Oficial GitHub: `https://github.com/VictorZainote/Bootcamp-3`
Status da Esteira CI/CD & Containerização: 100% Passed (pytest / GitHub Actions / Docker)

1. Síntese Executiva: A Arquitetura da Confiança

No desenvolvimento corporativo moderno assistido por Inteligência Artificial, garantir apenas que o código funcione localmente na máquina do desenvolvedor não é suficiente. A entrega final do sistema **TaskAnalyzer** consolida as três fases da jornada de Engenharia de Software do Bootcamp III:

Fase	Papel do Engenheiro	Entregáveis e Marcos de Governança
Fase 1 (10,0 pts)	Arquiteto de Especificações	Definição do System Design Document (SDD), contrato de dados em specs/ e regras de co
Fase 2 (15,0 pts)	Piloto de IA e Engenheiro QA	Geração supervisionada de src/task_analyzer.py, suíte pytest (tests/test_harness.py) e fluxo
Fase 3 (25,0 pts)	Especialista DevOps & Governança	Motor CI/CD (GitHub Actions), Containerização Isolada (Docker / Compose) e Auditoria Étic

2. Análise de Alucinações: O Firewall SDD e CONTEXT_RULES.md

Modelos de Linguagem de Grande Escala (LLMs) operam por predição probabilística de tokens, o que frequentemente induz alucinações estruturais, importação de bibliotecas não autorizadas e desvios silenciosos de contrato. No projeto **TaskAnalyzer**, a combinação do contrato executável (`specs/task_analyzer_spec.md` e `specs/requirements.yaml`) com as diretrizes restritivas de `CONTEXT_RULES.md` atuou como uma barreira determinística que neutralizou quatro vetores clássicos de alucinação:

Vetor de Risco (IA sem Governança)	Regra de Bloqueio (SDD + CONTEXT_RULES)	Resultado Obtido no Código Homologado
Importação de bibliotecas pesadas ou ext [illegible]	[illegible]	[illegible]
Alucinação de chaves no dicionário de sa [illegible]	[illegible]	[illegible]
Uso de ValueError genérico ou cálculo si [illegible]	[illegible]	[illegible]
Falha catastrófica por ZeroDivisionError a [illegible]	[illegible]	[illegible]

3. Homologação Humana: O Modelo 'Critical-in-the-Loop' na Prática

A governança ética aplicada neste projeto rejeita a automação cega (*Out-of-the-Loop*) e posiciona o engenheiro de software como filtro primário e homologador crítico (**Critical-in-the-Loop**). O ciclo de vida de cada incremento de código seguiu quatro estágios auditáveis:

Estágio 1 — Geração Orientada por Contexto: O agente de IA recebe exclusivamente prompts ancorados no SDD e no `CONTEXT_RULES.md` para propor a implementação de `analyze_tasks`.
Estágio 2 — Barreira SDD (Firewall de Contrato): O código gerado passa pela verificação estática de assinaturas, tipagem explícita (`list[dict[str, Any]] -> dict[str, Any]`) e pureza de dependências.
Estágio 3 — Validação Humana e QA Ativo: O engenheiro QA inspeciona criticamente os pontos de falha algorítmica — em especial: (a) proteção contra `ZeroDivisionError` quando `total_concluidas == 0` ou quando uma prioridade específica não possui entregas concluídas; (b) validação de anomalias temporais onde `data_conclusao < data_criacao` (inclusive em registros pendentes que contenham timestamp inválido ou `data_prazo < data_criacao`) exigindo `TaskValidationError`; e (c) rejeição de tipos booleanos disfarçados de inteiros em Python (`isinstance(val, bool)` em `prazo_estimado_horas`).

Estágio 4 — Integração Segura e Imutável: O código homologado só é integrado à branch principal após aprovação de 100% na suíte `pytest` em ambiente limpo (GitHub Actions e container Docker).

4. Sustentabilidade, Ética e Rastreabilidade de Prompts

A ética em sistemas construídos com auxílio de IA exige **rastreabilidade ponta a ponta** (saber exatamente quais instruções geraram cada artefato) e **sustentabilidade computacional** (uso de imagens enxutas como `python:3.11-slim` e algoritmos O(N) em passo único sem bibliotecas pesadas). Abaixo estão registrados os prompts auditáveis utilizados nas Fases 2 e 3:

Prompt #1 — Implementação do Core de Negócio (src/task_analyzer.py):

```
"Atue como engenheiro de software Python 3.11+. Utilizando estritamente as regras de
governança definidas em CONTEXT_RULES.md e o contrato técnico executável em
specs/task_analyzer_spec.md, implemente o módulo src/task_analyzer.py contendo a exceção
customizada TaskValidationError e a função analyze_tasks(tasks: list[dict[str, Any]]) ->
dict[str, Any]. Garanta 100% de cobertura de Type Hints, uso exclusivo da biblioteca padrão
do Python, validação temporal (data_conclusao < data_criacao disparando TaskValidationError)
e prevenção total contra ZeroDivisionError retornando 0.0 quando não houver tarefas
concluídas."
```

Prompt #2 — Implementação da Suíte de Testes (tests/test_harness.py):

```
"Com base nos Cenários de Aceite (Dado-Quando-Então) definidos em
specs/task_analyzer_spec.md, implemente a suíte de testes automatizados em
tests/test_harness.py utilizando pytest. Cubra integralmente: (1) Cenário 1 de sucesso
validando total_tarefas, total_concluidas, total_pendentes, tempo_medio_conclusao_horas
(geral, alta, media, baixa) e taxa_atraso_percentual; (2) Cenário 2 de erro validando o
disparo de TaskValidationError para datas inconsistentes (data_conclusao < data_criacao); e
(3) Casos de borda com lista vazia, apenas tarefas pendentes e validação de tipos inválidos."
```

Prompt #3 — Esteira CI/CD e Orquestração Docker (Fase 3):

```
"Seguindo a arquitetura final do repositório sdd-task-analyzer (Fase 3), estruture: (1) o
workflow .github/workflows/ci.yml com gatilhos push e pull_request nas branches main e
develop, executando em ubuntu-latest com Python 3.11, instalando requirements.txt e rodando
'pytest tests/test_harness.py -v'; e (2) o Dockerfile baseado na imagem oficial leve
python:3.11-slim e o docker-compose.yml para execução isolada e reprodutível da suíte de
testes com um único comando ('docker compose run --rm task-analyzer')."
```

5. Matriz de Conformidade Técnica e Ética (Fase 1 a Fase 3)

Dimensão Avaliada	Artefato no Repositório	Mecanismo de Verificação	Status
Especificação SDD	specs/system_design_document.pdf specs/requirements.yaml	Contrato de Entrada/Saída e BDD	**CONFORME**
Governança de IA	CONTEXT_RULES.md	Restrição de libs e tipagem 100%	**CONFORME**
Código Homologado	src/task_analyzer.py	Homologação Critical-in-the-Loop	**CONFORME**
Suíte de Testes QA	tests/test_harness.py	pytest tests/test_harness.py -v	**100% PASSED**
Motor CI/CD	.github/workflows/ci.yml	GitHub Actions (ubuntu-latest / Py 3.11)	**GREEN BUILD**
Containerização	Dockerfile + docker-compose.yml	Imagem isolada python:3.11-slim	**CONFORME**

6. Conclusão e Declaração de Responsabilidade Técnica

A arquitetura final do projeto **TaskAnalyzer** comprova que a produtividade proporcionada pela Inteligência Artificial Generativa só atinge maturidade de produção quando subordinada à engenharia de especificações (SDD), à homologação humana crítica (*Critical-in-the-Loop*) e à automação determinística via containers Docker e pipelines de Integração Contínua (CI/CD). Todo o código-fonte, testes e configurações encontram-se auditados, versionados e validados com 100% de aprovação.